

SMART ROUTE FINDER USING BFS, DFS, UCS

Submitted By: Neej Chetankumar Butani

Reg No.: 25BCE10485

Course: CSA2001

Abstract

This project presents a Smart Route Finder system that determines optimal paths between cities using classical Artificial Intelligence search algorithms. The system implements Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform Cost Search (UCS), along with visualization using graphs. The goal is to compare different search strategies and identify the most efficient route based on distance and traversal behavior.

Introduction

Route finding is a fundamental problem in Artificial Intelligence and graph theory. It is widely used in navigation systems, logistics, and transportation networks.

This project models cities as nodes and roads as weighted edges in a graph. Different search algorithms are applied to find paths between a source and a destination.

Objectives

- To understand graph-based search algorithms
- To implement BFS, DFS, and UCS
- To visualize graph traversal
- To compare algorithm performance

- To simulate real-world route planning

Technologies Used

- Python
- NetworkX (Graph handling)
- Matplotlib (Visualization)
- Heapq (Priority Queue for UCS)
- Collections (Deque for BFS)

Problem Statement

Given a set of cities and distances between them, find a path from a start city to a destination city using different search algorithms and determine the most optimal route.

Code

```

from collections import deque
import heapq
import networkx as nx
import matplotlib.pyplot as plt

# Graph with weights
graph = {
    'Bhopal': {'Indore': 190, 'Jabalpur': 330},
    'Indore': {'Ujjain': 55, 'Gwalior': 500},
    'Jabalpur': {'Gwalior': 450},
    'Ujjain': {'Gwalior': 450},
    'Gwalior': {}
}

# Convert to NetworkX graph
G = nx.DiGraph()
for node in graph:
    for neighbor, weight in graph[node].items():
        G.add_edge(node, neighbor, weight=weight)

# BFS
def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        node, path = queue.popleft()

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)

            for neighbor in graph[node]:

```

```

                queue.append((neighbor, path + [neighbor]))

    return None

# DFS
def dfs(graph, start, goal, path=[]):
    path = path + [start]

    if start == goal:
        return path

    for neighbor in graph[start]:
        if neighbor not in path:
            result = dfs(graph, neighbor, goal, path)
            if result:
                return result

    return None

# UCS
def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return cost, path

        if node not in visited:

```

```

        visited.add(node)
        for neighbor, weight in graph[node].items():
            heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor]))

    return None

# Visualization function
def draw_graph(path=None, title="Graph"):
    pos = nx.spring_layout(G)

    plt.figure()

    # Draw nodes & edges
    nx.draw(G, pos, with_labels=True, node_size=2000)

    # Draw edge weights
    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

    # Highlight path
    if path:
        edges = list(zip(path, path[1:]))
        nx.draw_networkx_edges(G, pos, edgelist=edges, width=3)

    plt.title(title)
    plt.show()

# Run
start = input("Enter the start point")
goal = input("Enter end point")

bfs_path = bfs(graph, start, goal)

```

```

    # Highlight path
    if path:
        edges = list(zip(path, path[1:]))
        nx.draw_networkx_edges(G, pos, edgelist=edges, width=3)

    plt.title(title)
    plt.show()

# Run
start = input("Enter the start point")
goal = input("Enter end point")

bfs_path = bfs(graph, start, goal)
dfs_path = dfs(graph, start, goal)
ucs_cost, ucs_path = ucs(graph, start, goal)

print("BFS Path:", bfs_path)
print("DFS Path:", dfs_path)
print("UCS Path:", ucs_path, "Cost:", ucs_cost)

# Visualize
draw_graph(bfs_path, "BFS Path")
draw_graph(dfs_path, "DFS Path")
draw_graph(ucs_path, "UCS Path (Optimal)")

```

System Design

Graph Representation

The graph is represented using an adjacency list where:

- Nodes represent cities
- Edges represent roads
- Weights represent distances

Example:

Bhopal → Indore (190)

Bhopal → Jabalpur (330)

Algorithms Used

Breadth-First Search (BFS)

- Explores level by level
- Uses queue
- Finds shortest path in terms of edges

Advantages:

- Complete and optimal (for unweighted graphs)

Disadvantages:

- High memory usage

Depth-First Search (DFS)

- Explores depth-wise
- Uses recursion/stack

Advantages:

- Low memory usage

Disadvantages:

- Not optimal
- May get stuck in deep paths

Uniform Cost Search (UCS)

- Expands lowest cost path first
- Uses priority queue

Advantages:

- Finds optimal path (weighted graphs)

Disadvantages:

- Slower than BFS

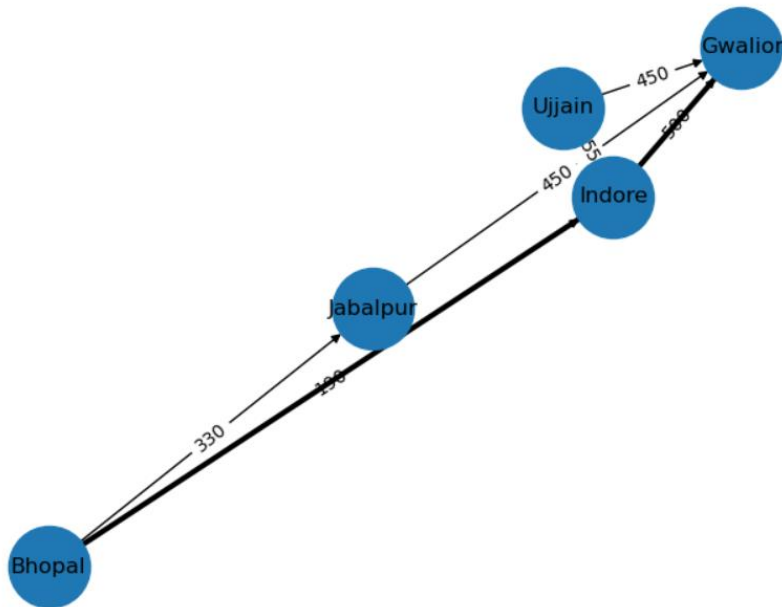
Implementation**Key Functions:**

- bfs() – finds path using queue
- dfs() – recursive depth search
- ucs() – cost-based search using heap
- draw_graph() – visualizes graph and path

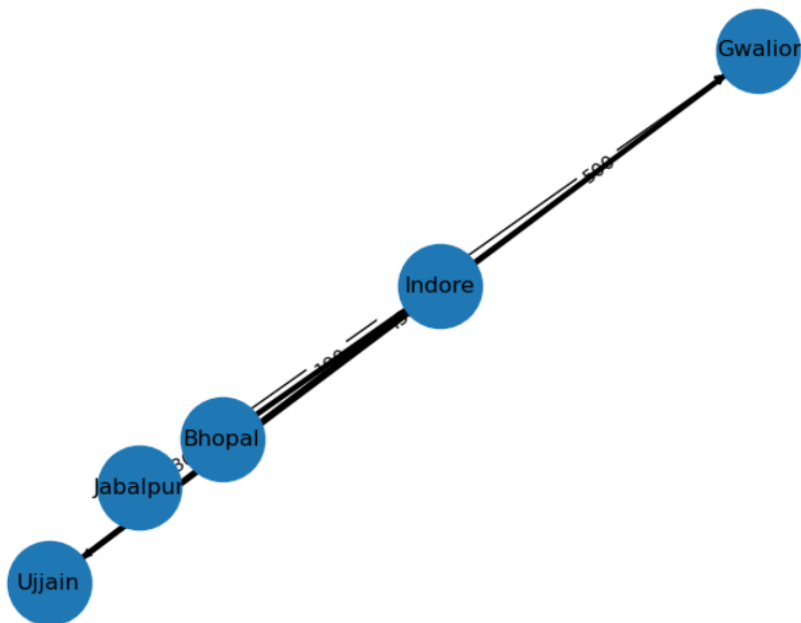
Output

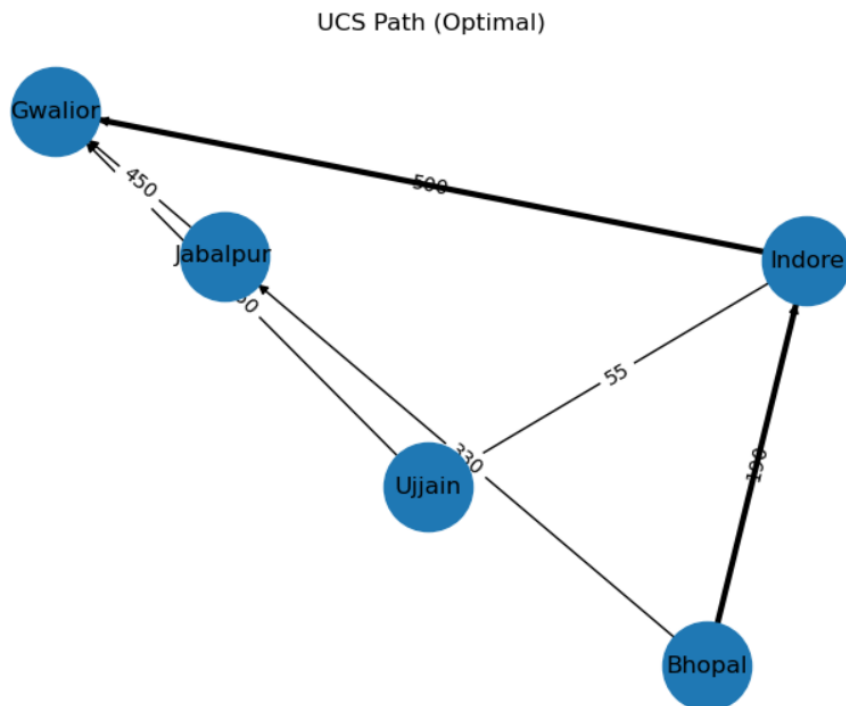
Enter the start point Bhopal
Enter end point Gwalior
BFS Path: ['Bhopal', 'Indore', 'Gwalior']
DFS Path: ['Bhopal', 'Indore', 'Ujjain', 'Gwalior']
UCS Path: ['Bhopal', 'Indore', 'Gwalior'] Cost: 690

BFS Path



DFS Path





Comparison Table

Algorithm		Optimal	Uses Weights	Data Structure	Use Case
BFS	Yes (unweighted)	No	No	Queue	Shortest path (steps)
DFS	No	No	No	Stack/Recursion	Exploration
UCS	Yes	Yes	Yes	Priority Queue	Cost-based routing

Applications

- GPS Navigation Systems
- Google Maps-like routing
- Delivery optimization
- Robotics path planning
- Network routing

Advantages of Project

- Easy to understand
- Visual representation
- Real-world relevance
- Multiple algorithm comparison

Limitations

- Uses static graph (no real-time data)
- No traffic consideration
- Limited number of nodes

Future Enhancements

- Integration with real maps (OpenStreetMap)
- Add traffic conditions
- Implement A* with real heuristics
- Web-based interface using Streamlit
- Multi-route suggestions

Conclusion

The Smart Route Finder demonstrates how different AI search algorithms behave in solving routing problems. UCS provides the most optimal solution for weighted graphs, while BFS and DFS help understand traversal strategies. This project forms a strong foundation for building real-world navigation systems.

References

- Python Documentation

- [NetworkX Documentation](#)
- [GeeksforGeeks \(Search Algorithms\)](#)